

CSED 312 Project 2 Final Report

김지현(20220302) 전현빈(20220259)

Table of Contents:

- 0. Overall project design
 - 1. Argument Passing
 - Implementation and Improvement from the previous design
 - Difference from design report
 - 2. System Calls
 - Implementation and Improvement from the previous design
 - Difference from design report
 - 3. Process Termination Messages
 - Implementation and Improvement from the previous design
 - Difference from design report
 - 4. Denying Writes to Executables
 - Implementation and Improvement from the previous design
 - Difference from design report
- 5. Overall Limitations
- 6. Overall discussion
- 7. Result

Overall Project Design

이번 프로젝트 2에서는 위의 목차에 1~4에 해당하는 네 가지 task가 있다. 각각의 task를 실행하기 위해 구현한 함수들을 설명하기 전에, 전체적인 User Program의 작동 방식에 대해 설명하겠다. 부모와 자식 프로세스 간의 관계를 저장하고 각 프로세스가 사용하는 리소스(추후 설명될 파일 포함)를 관리하기 위해 thread 구조체에 변수들을 추가하였다.

```
struct thread
{
...
#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;          /* Page directory. */

    struct thread *parentThread;
    struct list_elem childElem;
    struct list childList;
```

```

    bool isLoad;
    bool isExit;

    struct semaphore semaWait;
    struct semaphore semaExec;
    int exitCode;

    struct file **fileTable;
    struct file *fileExec;
    int fileCnt;
#endif
...
};

```

위의 요소들은 수업에서 배운 process control block (PCB)에 해당하는 정보들로 각 프로세스마다 저장된다. 이때, 사용자 프로그램을 실행할 때 위의 요소들이 필요하기 때문에 `#ifdef`를 사용해 `userprog` 매크로가 설정되었을 때만 생성되도록 구현하였다.

위에서부터 순서대로 해당 스레드가 사용하는 가상 메모리 공간인 "page directory", 부모 프로세스를 가리키는 스레드 포인터 "parentThread", 자식 프로세스 리스트에서 해당 스레드를 나타내기 위한 "childElem", 현재 프로세스가 생성한 자식 프로세스들의 링크드 리스트인 "childList"가 있다. 또한, 이후 과정에서 사용될 두 가지 불리언 변수인 `isLoad`와 `isExit` 그리고 두 가지 세마포어 `semaWait`과 `semaExec`이 있다. 마지막으로, 스레드가 종료될 때 반환하는 "exit code"가 있다.

다음으로는 파일 구조와 관련된 변수들이다. 먼저 "fileTable"은 file descriptor마다 열려 있는 파일에 대한 정보를 저장하며, `fileExec`은 현재 실행 중인 파일을 가리킨다. 마지막으로 `fileCnt`는 현재 프로세스가 열어 놓은 파일의 개수를 나타내는 정수 변수이다.

```

tid_t
thread_create (const char *name, int priority,
               thread_func *function, void *aux)
{
    ...
    // #ifdef USERPROG
    thread_userprog_init(t);
    t->fileTable = palloc_get_page(PAL_ZERO);
    if(t->fileTable == NULL)
    {
        return TID_ERROR;
    }
    t->fileCnt = 2;
    // #endif
    ...
}

void
thread_userprog_init(struct thread *t)
{
    t->parentThread = thread_current();
}

```

```

    list_push_back(&(thread_current()->childList), &(t->childElem));

    t->isExit = false;
    t->isLoad = false;

    sema_init(&(t->semaWait), 0);
    sema_init(&(t->semaExec), 0);
}

```

그리고 앞서 설명한 것처럼, `thread` 구조체에서 정의된 새로운 객체들을 `thread_create` 함수에서 초기화하였다. 이와 비슷하게 `#ifdef`를 이용하여 `userprog` 매크로에 따라 실행 여부를 확인하도록 구현하였다. PCB와 관련된 변수들은 `thread_userprog_init` 함수 내부에서 따로 초기화하였다. 새로운 스레드를 생성하는 것이기 때문에 현재 실행 중인 스레드를 `parentThread`로 설정하고, `parentThread`의 `childList`에 `t` 스레드를 추가하였다. 또한, 기타 boolean 변수들과 세마포어들을 초기화하였다.

파일과 관련된 변수들도 초기화하였는데, 먼저 `fileTable`에 메모리를 할당한 후 `PAL_ZERO` 플래그를 사용하여 할당된 메모리를 0으로 초기화하였다. 일반적으로 프로세스가 실행될 때는 `stdin`과 `stdout`이 있기 때문에 `fileCnt`를 2로 설정하였다. 이와 관련된 내용은 아래의 `syscall` 파일 부분에서 더 자세히 설명할 것이다.

1. Argument Passing

Implementation & Improvement from the previous design

첫 번째 과제는 argument passing이다. argument는 어떤 함수를 호출할 때 넘겨주는 인자로, 기존의 `process_execute` 함수는 argument(인자)를 넘겨주지 않는 방식으로 구현되어 있다. 그러나 Pintos 문서에 설명된 바와 같이 "grep foo bar"와 같은 명령어를 실행하려면 "foo"와 "bar"라는 인자를 넘겨주면서 "grep" 함수를 호출해야 한다. 이를 위해, 명령어를 " "(공백)을 기준으로 parsing해야 한다.

따라서 `process_execute` 함수에서 위의 명령어를 parsing하는 함수를 추가하였다.

(또한, `process_execute` 함수에서 앞서 설명한 `thread_create` 함수를 호출하며, 이 함수의 인자 중 하나로 `start_process`가 넘겨지는데, 이 함수에 대한 설명은 아래에 이어서 설명할 것이다.)

```

int make_argv(char **argv, char *file_name){
    char *cmd_1st;          /* file name */
    char *cmd_remainder;    /* remainder */
    char *iterS = strtok_r (file_name, " ", &cmd_remainder);
    char *iterE = NULL;
    int argc = 0;

    for (cmd_1st = iterS; cmd_1st != iterE; cmd_1st = strtok_r (NULL, " ",
&cmd_remainder)){
        argv[argc] = cmd_1st;
        argc++;
    }
    return argc;
}

```

함수 `make_argv`는 이름 그대로 `argv`를 build한다. 즉, 입력된 `file_name`을 공백 " "을 기준으로 하여 파싱하고 각 토큰(분리된 단어)들을 `argv`에 순서대로 저장한 후 해당 토큰의 갯수 `argc`를 return한다. 이는 일반적으로 커맨드 라인을 토큰화 하여 각 인자를 배열에 저장하고 인자의 갯수를 계산하는 역할을 한다. 이는 argument passing 전체를 구현함에 있어 가장 기초가 되는 단계를 구현한다. 처음 `file_name`은 `strtok_r`에 의해 파싱되며 이때 첫번째 토큰이 `cmd_1st`에 저장된다. 그리고 해당 토큰을 제외한 나머지 덩어리는 `cmd_remainder`에 저장된다. 즉 예를 들어 "grep foo bar"의 경우 "grep"은 `cmd_1st`에, "foo bar"는 `cmd_remainder`에 저장된다. 이후 반복문을 사용하여 `cmd_1st`는 남은 `cmd_remainder`의 첫번째 토큰을 계속해서 가르키게 되는데, 이때 `iterS`는 전체 커맨드(`file_name`)의 가장 첫번째 토큰(반복문 시작 토큰)이며 `iterE`는 NULL(for 루프 종료 조건 설정)이다. 이처럼 for루프에서는 `file_name`을 순차적으로 공백 기준 토큰화하여 `cmd_1st`에 저장하고 `argv`배열에 순차적으로 삽입한다. (`argv[argc]`에 저장 후 `argc`증가)

이때, Pintos 문서에 설명된 'lib/string.h' 파일의 `strtok_r` 함수를 사용하여 구현하였다. 각 인자는 `char** argv`에 저장되며, `int argc`는 인자의 총 개수를 나타낸다.

```
static void
start_process (void *file_name_)
{
    ...
    int argc;
    char** argv;
    argv = palloc_get_page(0);
    argc = make_argv(argv, fn_copy); //

    success = load (argv[0], &if_.eip, &if_.esp);

    thread_current()->isLoad = success;
    if(success){ //new
        cmd_stack_build(argv, argc, &if_.esp);
    }

    sema_up(&thread_current()->semaExec);
    palloc_free_page (fn_copy);
    palloc_free_page (argv);
    ...
}
```

`start_process` 함수는 user program을 새로운 process로 로드하고 실행하는 역할을 한다.

`start_process`에서는 `argc`, `argv`를 선언 후, 입력받은 `file_name_`에 저장된 커맨드를 앞서 구현한 `make_argv`를 이용해 파싱하고 저장한다. 기존구현에서는 `load`함수에 `file_name`을 전달하였으나 이제는 `argv[0]`을 전달함으로써 커맨드의 명령어만을 올바르게 의도한대로 전달할 수 있다. `load`함수는 boolean으로, 메모리에 성공적으로 로드했는지에 대한 여부를 반환하여 `success`에 저장한다. 따라서 현재 thread의 `isLoad`는 `success`와 같게 해준다. `load`의 구현은 나중에 설명한다.

또한 `success == true`인 경우, 프로그램을 실행해야하므로 필요한 인자를 user stack에 push해야하고 이를 수행하는 함수를 아래와 같이 구현했다.

```
void cmd_stack_build(char **argv, int argc, void **esp){
    /* argv[i][data] push */
```

```
int len = 0;
for (int i = argc - 1; i >= 0; i--) {
    // 큰 index부터 차례로 push해야 한다.
    len = strlen(argv[i]); // len은 명령어의 단어 길이
    *esp -= len + 1;
    strcpy(*esp, argv[i], len + 1);
    argv[i] = *esp;
}

/* Align Stack */
// 만약 *esp가 4의 배수가 아니면
if ((uint32_t)(*esp) % 4 != 0) {
    // *esp를 가장 가까운 4의 배수로 내림
    *esp = *esp - ((uint32_t)(*esp) % 4);
}

/* NULL push */
*esp -= 4;
**(uint32_t **)esp = 0;

/* argv[i] push */
for (int i = argc - 1; i >= 0; i--) {
    *esp -= 4;
    **(uint32_t **)esp = argv[i];
}

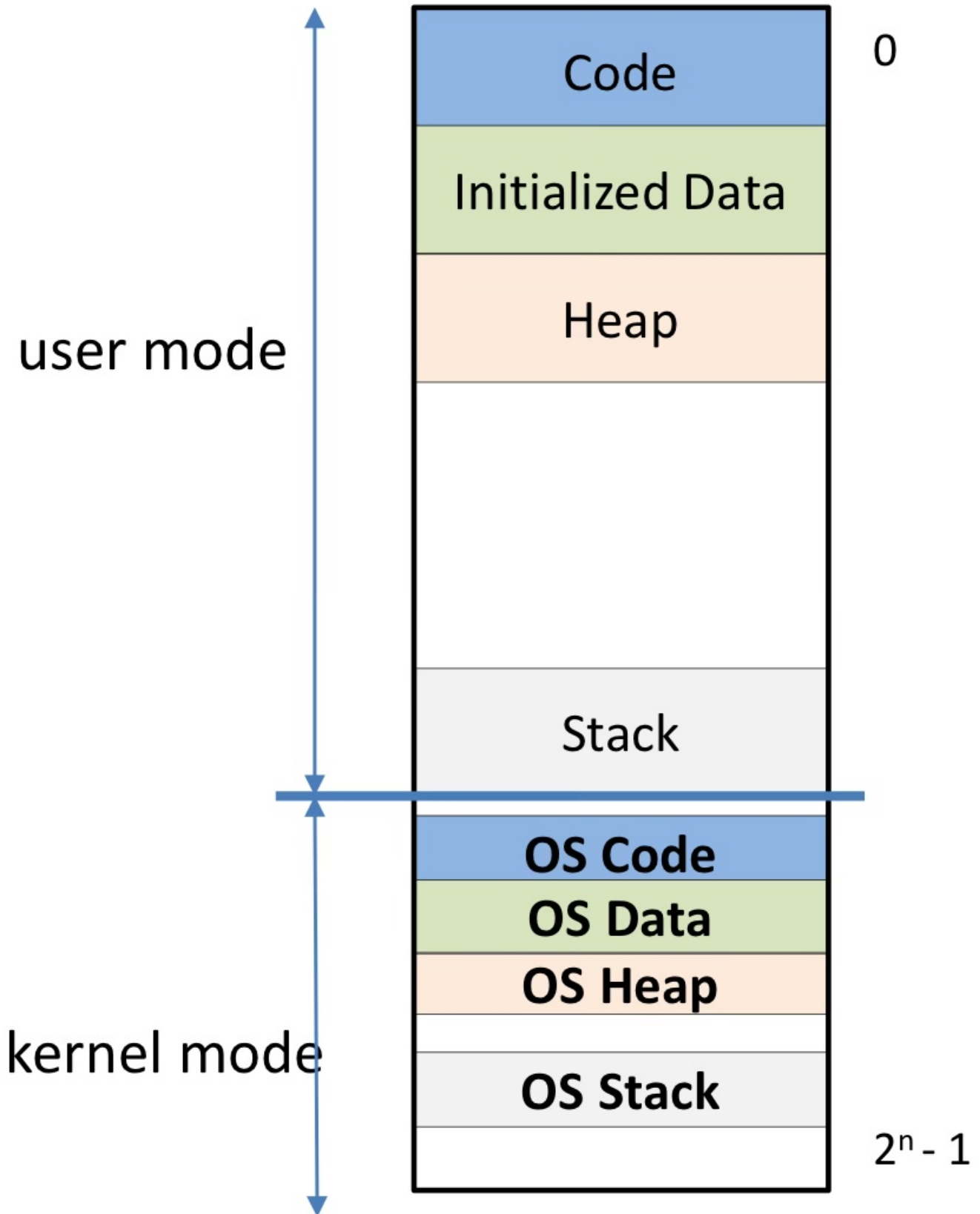
/* argv push */
*esp -= 4;
**(uint32_t **)esp = *esp + 4;

/* argc push */
*esp -= 4;
**(uint32_t **)esp = argc;

/* return point push */
*esp -= 4;
**(uint32_t **)esp = 0;
}
```

위의 함수는 메모리에 stack을 적절히 구성하는 기능을 수행한다. pintos document에서는 아래와 같이 stack을 구성할것을 명시하고있다.

Address	Name	Data	Type
0xbfffffffcc	argv[3][...]	'bar\0'	char[4]
0xbfffffff8	argv[2][...]	'foo\0'	char[4]
0xbfffffff5	argv[1][...]	'-l\0'	char[3]
0xbfffffed	argv[0][...]	'/bin/ls\0'	char[8]
0xbfffffec	word-align	0	uint8_t
0xbfffffe8	argv[4]	0	char *
0xbfffffe4	argv[3]	0xbfffffffcc	char *
0xbfffffe0	argv[2]	0xbfffffff8	char *
0xbfffffdc	argv[1]	0xbfffffff5	char *
0xbfffffd8	argv[0]	0xbfffffed	char *
0xbfffffd4	argv	0xbfffffd8	char **
0xbfffffd0	argc	4	int
0xbfffffcc	return address	0	void (*)()



따라서 $\text{index} = \text{argc} - 1$ 부터 $\text{index} = 0$ 까지 for 루프를 이용하여 $\text{argv}[\text{index}][\text{data}]$ 를 push하고, 메모리 접근 효율을 위해 align을 맞추기 위해 0을 조건에 맞게 push한다. 이후 NULL을 푸시하며 다음에는 $\text{argv}[\text{index}]$ 를 푸시한 후 argv , argc , return point를 푸시함으로써 stack build를 종료한다.

상기한 함수들의 구현과 사용을 통해, 입력된 커맨드를 파싱하여 저장하고 적절히 스택을 구성하며 메모리에 로드하여 유저프로그램을 실행시키는 과정을 준비할 수 있다.

```

void
process_exit (void)
{
    struct thread *cur = thread_current ();
    uint32_t *pd;

    for(int i = 2; i < cur->fileCnt; i++)
    {
        sys_close(i);
    }

    // 이제 전부 해제
    palloc_free_page(cur->fileTable); // 스레드가 가지고 있던 테이블 해제
    file_close(cur->fileExec); // 파일을 닫는다.
    ...
}

```

유저 프로세스를 종료한다는 것은, 파일을 close하는 과정을 포함해야한다. 따라서 프로세스의 종료를 구현하는 `process_exit`에 위위와 같이 현재 execute중인 file에 대해 `file_close`함수를 이용해 close해주어야 한다. 따라서 execute되고 있던 unwritable file은 (i.e., `cur->fileExec`) 쓰기가가능해진다.

```

bool
load (const char *file_name, void (**eip) (void), void **esp)
{
    struct thread *t = thread_current ();
    struct Elf32_Ehdr ehdr;
    struct file *file = NULL;
    off_t file_ofs;
    bool success = false;
    int i;

    /* Allocate and activate page directory. */
    t->pagedir = pagedir_create ();
    if (t->pagedir == NULL)
        goto done;
    process_activate ();

    //////////////////////////////////////
    // 여기 아래는 file 접근이므로 FileLock!
    lock_acquire(&FileLock); // 파일락 획득 후 내려감
    /* Open executable file. */
    file = filesys_open (file_name);
    if (file == NULL)
    {
        lock_release(&FileLock); // 파일 못열었으니까 파일락 반환, 실패 메시지 출력
        printf ("load: %s: open failed\n", file_name);
        goto done;
    }
}

```



```

// 파일 획득 성공시
t->fileExec = file; // 스레드에서 실행하는 fileExec는 file임.
lock_release(&FileLock);
////////////////////////////////////

/* Read and verify executable header. */
if (file_read (file, &ehdr, sizeof ehdr) != sizeof ehdr
    || memcmp (ehdr.e_ident, "\177ELF\1\1\1", 7)
    || ehdr.e_type != 2
    || ehdr.e_machine != 3
    || ehdr.e_version != 1
    || ehdr.e_phentsize != sizeof (struct Elf32_Phdr)
    || ehdr.e_phnum > 1024)
{
    printf ("load: %s: error loading executable\n", file_name);
    goto done;
}

/* ... 기존 구현에 해당하는 부분으로, 중략 ... */

/* Set up stack. */
if (!setup_stack (esp))
    goto done;

/* Start address. */
*eip = (void (*) (void)) ehdr.e_entry;

success = true;

done:
/* We arrive here whether the load is successful or not. */
// file_close (file);
/* 없어짐!!! */
return success;
}

```

프로세스를 실행시키기 위해 위의 과정에서 파싱하고 저장한 커맨드를 바탕으로 올바르게 file에 접근하여 memory에 load해야하는데, 이 과정은 load함수에 의해 구현된다. **load**는 memory로드를 시도하고 성공여부를 boolean으로 return 한다.

즉, load함수는 공유자원인 file에 접근한다. 따라서 단일한 file에의 접근을 atomic하게 구현해야하므로 파일 접근을 **FileLock**이라는 lock을 이용해 보호했다. FileLock acquire후 file load를 시도하여 실패하면 release 하고 실패 메시지를 출력하며, file획득에 성공한 경우, 해당 file 은 execute되는 파일이므로 **t->fileExec = file;**를 통해 현재 실행되는 파일임을 명시한다. 이후 해당 file이 실행되는 도중 수정되어선 안되므로 **file_deny_write(file)**를 통해 파일이 다른 writer에 의해 수정되지 못하도록 보호해야 하는데 이 과정은 아래에서 다시 설명할 예정이다. 이 과정 이후에는 파일에 대한 접근과 처리를 완료하였으므로 FileLock은 release 해주어도 된다.

또한 file_close는 프로세스 종료시 수행되는 **process_exit**에서 처리하므로 load에서 호출될 필요가 없으므로 기존구현에서 주석처리로 제거하였다.

마지막으로 사용자 프로그램이 올바른 메모리 접근을 하는지 확인하기 위해 `exception.c` 의 `page_fault` 함수를 수정해주었다.

```
static void
page_fault (struct intr_frame *f)
{
    ...
    if(not_present || is_kernel_vaddr(fault_addr) || !user){
        sys_exit(-1);
    }
    ...
}
```

먼저, `not_present`를 통해 접근하려는 페이지가 메모리에 있는지 확인해준다. 다음으로, `is_kernel_vaddr` 함수를 사용하여 커널 메모리에 접근하려고 하는지를 검사한다. 마지막으로, `!user`를 통해 해당 함수 호출이 사용자 모드에서 발생했는지 확인해주었다. 정리하면, 페이지가 메모리에 없거나, 페이지 폴트가 커널 메모리에서 발생했거나, 사용자 모드에서 페이지 폴트가 발생했다면 (셋 중 하나라도 true 라면), `sys_exit` 함수 호출을 통해 해당 프로세스를 종료하도록 했다.

이렇게 첫 번째 과제인 argument passing에서 요구하는 목적을 달성할 수 있다.

Difference from design report

`strtok_r` 함수를 사용하여 구현한다는 것은 랩 시간과 Pintos 문서에서도 설명되었기 때문에 명령어 parsing 과정을 어려움 없이 진행할 수 있었다. 그러나 `start_process` 함수에서 앞서 parsing한 인자들과 인자 개수, return address 등을 user stack에 추가하는 함수에 대한 구체적인 설계는 하지 않았다. 하지만 수업에서 배운 것처럼 user stack은 가장 높은 esp 에서부터 4byte 씩 감소하면서 순서대로 값이 저장되어야 하는지에 대한 개념을 바탕으로 구현을 완료하였다.

2. System Calls

Implementation & Improvement from the previous design

두 번째 과제는 system call을 구현하는 것이었다. system call은 user program이 OS 기능에 접근하기 위해 사용하는 함수들로 크게 두 가지로 나눌 수 있다. process 작동에 대한 기능들과 파일 접근에 대한 기능이다. 먼저 process 작동에 대한 기능으로는 `halt`, `exit`, `exec`, 그리고 `wait`가 있으며, 파일 접근에 대한 기능으로는 `create`, `remove`, `open`, `filesize`, `read`, `write`, `seek`, `tell`, 그리고 `close`가 있다.

```
void sys_halt(void)
{
    shutdown_power_off();
}
```

먼저 `sys_halt`는 Pintos 문서에 설명된 대로 `shutdown_power_off` 함수를 호출하는 동작만 수행하면 된다.

```
void sys_exit(int status)
{
    thread_current()->exitCode = status;
    thread_exit();
}
```

`sys_exit` 함수는 현재 실행 중인 사용자 프로그램을 종료하고 커널에 `status`를 반환하는 함수이다. 현재 프로세스의 부모 프로세스가 `wait` 중이라면, 반환되는 것은 이 `status` 값이다. 즉, `status`는 종료 코드(`exit` 코드)와 같으며, 일반적으로 `0`은 현재 프로세스가 성공적으로 종료되었음을 나타내고, `0`이 아닌 값은 오류가 발생했음을 의미한다. 해당 함수는 `thread_exit` 함수 호출을 통해 실제로 프로세스를 종료시킬 수 있다. 위의 함수에서 현재 스레드의 `exitCode`에 `status` 값을 저장한 뒤, `thread_exit` 함수를 통해 해당 스레드를 종료시킨다.

```
pid_t sys_exec(const char *file)
{
    if(!verify_mem_address(file)){
        sys_exit(-1);
    }

    pid_t pid = process_execute(file);
    if (pid == -1) return -1;

    struct thread *child = getChild(pid);
    sema_down(&child->semaExec);

    return (child->isLoad) ? pid : -1;
}
```

`sys_exec`는 `file`로 전달된 `name`을 가진 실행 파일을 실행시키는 함수로, 새로운 프로세스의 `pid`(program id)를 반환한다. 해당 프로그램이 실행되지 못하면 `-1`을 반환한다. 먼저 `verify_mem_address` 함수를 통해 `file`이 올바른 사용자 주소를 가리키는지 확인한 후, `process_execute` 함수를 호출하여 자식 프로세스를 생성한다. 이때, `process_execute` 함수의 반환값이 `-1`이면 실행되지 못한 것이므로, `sys_exec`도 `-1`을 반환하도록 한다.

```
struct thread *getChild(pid_t pid)
{
    struct list_elem *e;
    struct list *childList = &thread_current()->childList;
    struct list_elem *iterS = list_begin(childList);
    struct list_elem *iterE = list_end(childList);
    struct thread *retrunThread = NULL;

    for (e = iterS; e != iterE; e = list_next (e))
    {
        struct thread *t = list_entry(e, struct thread, childElem);
        if(t->tid == pid){
```

```

    retrunThread = t;
}
}
return retrunThread;
}

```

그리고 위의 `sys_exec` 함수를 보면 `getChild` 함수 호출을 통해 자식 프로세스를 받아온 뒤 세마포어를 이용해 자식 process 에서 user program 이 load 될때까지 wait 하는 instruction 을 추가하였다. 이때, `getChild` 함수를 보면 현재 thread 의 자식 프로세스들을 for loop 를 통해 순회하며 만약 해당 tid 가 pid 와 동일하다면 해당 thread 를 반환하도록 하였다.

여기서 설정한 `sema_down` 은 위에서 설명한 `start_process` 에서 `isLoad` 를 true 로 바꿔주며 `sema_up` 또 함께 진행된다.

```

int sys_wait(pid_t pid)
{
    return process_wait(pid);
}

```

이 함수는 부모 프로세스가 자식 프로세스가 종료될 때까지 기다리며, 자식의 `exit status`를 저장하고 `pid`를 반환받는다. 만약 자식 프로세스가 종료되지 않았는데 커널에 의해 종료되었다면, `-1`을 반환하여 부모 프로세스가 `exit status`를 받을 수 있도록 한다. 이렇게 구현함으로써, 커널에 의해 자식 프로세스가 종료되었음을 부모 프로세스가 확인할 수 있다. 이 과정은 `process_wait` 함수 호출을 통해 구현하였다.

```

int
process_wait (tid_t child_tid)
{
    struct thread *parentThread = thread_current();
    struct thread *childThread = getChild(child_tid);
    if (childThread==NULL)
    {
        return -1;
    }
    int returnCode;

    sema_down(&childThread->semaWait); // make ATOMIC
    returnCode = childThread->exitCode;
    list_remove(&(childThread->childElem));
    palloc_free_page(childThread); // make ATOMIC

    return returnCode;
}

```

`sys_wait`를 구현하기 위해 사용된 `process_wait` 함수는 먼저 자식 프로세스의 존재 여부를 확인한다. 만약 자식 프로세스가 없다면 (즉, `NULL` 이라면) 대기하지 않고 종료하기 위해 `-1`을 반환한다. 자식 프로세스가 존재하는 경우에는 자식 프로세스가 종료될 때까지 세마포어를 사용해 대기한 후, 자식 프로세스의 `exitCode`를 저장한다. 이후 자식 프로세스의 메모리를 해제하고 동기화를 해제한 다음, 저장한 `exitCode`를 반환한다.

이후의 함수들은 파일 접근에 관한 기능들이다.

```
bool sys_create(const char *file, unsigned initial_size)
{
    if(!verify_mem_address(file)||file==NULL)
    {
        sys_exit(-1);
    }
    return filesys_create(file, initial_size);
}
```

sys_create 함수는 "file"이라는 이름과 "initial_size" 크기를 가진 새로운 파일을 생성하는 함수로, 파일 생성 성공 여부에 따라 boolean 값을 반환한다. 파일을 생성하는 것과 파일을 여는 것은 다른 개념이며, 이는 아래의 **open** 함수에서 설명할 것이다. 이 함수에서는 먼저 **verify_mem_address** 함수를 통해 인자로 받은 **file**이 올바른 주소에 있는지 확인하고, **file**이 **null**이 아닌지 검사한 후 **filesys_create** 함수를 호출하여 새로운 파일을 생성한다. 만약 두 가지 조건 중 하나라도 충족되지 않으면, 위에서 설명한 **sys_exit** 함수를 호출하여 오류를 처리한다.

```
bool sys_remove(const char *file)
{
    if(!verify_mem_address(file)){
        sys_exit(-1);
    }

    return filesys_remove(file);
}
```

sys_remove 함수는 파일이 열려 있는지 여부와 상관없이 "file" 이름을 가진 파일을 삭제하는 역할을 하며, 파일 삭제 성공 여부를 반환한다. 이 함수는 열려 있는 파일을 삭제하더라도 해당 파일을 닫지 않는다. 이 함수에서도 **verify_mem_address** 함수를 사용하여 **file**이 유효한 주소를 가지는지 확인한 후, **filesys_remove** 함수를 호출하여 파일 삭제를 처리한다.

```
struct lock FileLock; // make file access ATOMIC
...
int sys_open(const char *file)
{
    if(!verify_mem_address(file)){
        sys_exit(-1);
    }

    lock_acquire(&FileLock);
    struct file *f = filesys_open(file);

    if (f == NULL) {
        lock_release(&FileLock);
        return -1;
    }
}
```

```
int fd = thread_current()->fileCnt;
thread_current()->fileTable[thread_current()->fileCnt++] = f;
lock_release(&FileLock);

return fd;
}
```

`sys_open` 함수는 "file"이라는 이름을 가진 파일을 여는 역할을 한다. 파일이 열리면 파일 디스크립터에 해당하는 양의 정수를 반환하고, 열리지 않으면 `-1`을 반환한다. 여기서 파일 디스크립터는 표준 입력 파일일 때 `0`, 표준 출력 파일일 때 `1`을 갖는다. 각 프로세스가 파일을 열 때마다 새로운 파일 디스크립터 번호가 할당되며, 이 파일 디스크립터는 각 프로세스별로 존재하며 자식 프로세스에는 전달되지 않는다. 또한, 동일한 파일을 여러 번 열 경우 매번 새로운 파일 디스크립터가 반환되므로, 각 파일 디스크립터는 별도로 닫아주어야 한다.

이 함수에서는 `verify_mem_address` 함수를 통해 유효한 주소인지 확인한 후 `filesys_open` 함수를 사용하여 파일을 연다. 여러 프로세스가 동시에 동일한 파일을 여는 것을 방지하기 위해 `FileLock`을 사용하여 `lock_acquire`와 `lock_release`로 접근을 제어하였다. 파일이 성공적으로 열리면 해당 파일을 `fileTable`에 넣고, 현재 스레드의 `fileCnt`를 1 증가시킨다.

```
int sys_filesize(int fd)
{
    struct file *f = (fd < thread_current()->fileCnt) ? thread_current()->fileTable[fd] : NULL;

    return (f != NULL) ? file_length(f) : -1;
}
```

`sys_filesize` 함수는 `fd`라는 파일 디스크립터를 가진 파일의 크기를 바이트 단위로 반환한다. 이 함수는 `file_length` 함수를 사용하여 파일의 크기를 계산하고 반환한다.

```
int sys_read(int fd, void *buffer, unsigned size)
{
    if (fd < 0 || fd >= thread_current()->fileCnt || !verify_mem_address(buffer)) {
        sys_exit(-1);
    }

    int read_size = 0;
    lock_acquire(&FileLock);

    if (fd == 0) {
        while (read_size < size && ((char *)buffer)[read_size] != '\0') {
            read_size++;
        }
    } else {
        struct file *f = thread_current()->fileTable[fd];

        if (f == NULL) {
            lock_release(&FileLock);
        }
    }
}
```

```

        sys_exit(-1);
    }
    read_size = file_read(f, buffer, size);
}

lock_release(&FileLock);
return read_size;
}

```

`sys_read` 함수는 `fd`라는 파일 디스크립터를 가진 파일에서 "size" 바이트만큼 읽어와 `buffer`에 저장하는 역할을 한다. 실제로 읽은 바이트 수를 반환하며, 읽을 수 없는 경우 `-1`을 반환한다. 예를 들어, `fd`가 `0`인 경우 이는 표준 입력이므로 `input_getc` 함수를 사용하여 키보드 입력을 읽어온다. 이를 위해 `while` 루프를 사용하여 `read_size`를 1씩 증가시키도록 구현하였다. 그 외의 경우에는 `file_read` 함수를 호출하여 `read_size`를 계산하였다. 이 과정 또한 동시에 일어나지 않도록 `lock`을 사용하여 원자적으로 진행되도록 하였다.

```

int sys_write(int fd, const void *buffer, unsigned size)
{
    for (int i = 0; i < size; i++) {
        if (!verify_mem_address(buffer + i)) {
            sys_exit(-1);
        }
    }

    if (fd < 1 || fd >= thread_current()->fileCnt) {
        sys_exit(-1);
    }

    int write_size = 0;
    lock_acquire(&FileLock);

    if (fd == 1) {
        putbuf(buffer, size);
        write_size = size;
    } else {
        struct file *f = thread_current()->fileTable[fd];

        if (f == NULL) {
            lock_release(&FileLock);
            sys_exit(-1);
        }

        write_size = file_write(f, buffer, size);
    }

    lock_release(&FileLock);
    return write_size;
}

```

`sys_write` 함수는 `fd`라는 파일 디스크립터를 가진 파일에 "size" 바이트만큼 `buffer`에 저장된 내용을 작성하는 역할을 하며, 실제로 작성된 바이트 수를 반환한다. 일반적으로 기존 파일 크기보다 더 많은 내용을 작성할 경우 파일 크기가 증가하지만, Pintos의 기본 파일 시스템은 고정 크기 파일로 구현되어 있어 파일의 끝(end of file, EOF)에 도달할 때까지 작성된 바이트 수를 반환한다.

먼저, `buffer`에서 `size`만큼의 메모리가 유효한 사용자 주소인지 확인하기 위해 `for` 루프를 통해 `buffer`의 각 바이트를 검사한다. 이를 위해 `verify_mem_address` 함수를 사용하여 `buffer + i` 주소가 유효한지 1바이트씩 확인해주었다. 이후, 주어진 파일 디스크립터가 유효한지 확인하였다. 쓰기 작업이므로 `fd`는 1 이상이어야 하며, 현재 스레드의 파일 개수(`fileCnt`)보다 크면 안된다. 두 조건 중 하나라도 만족하지 않으면 `sys_exit` 함수를 호출하여 오류를 처리해주었다.

예를 들어, `fd`가 1인 경우 이는 표준 출력이므로 `put_buf()` 함수를 사용하여 콘솔에 `buffer`에 저장된 값을 출력한다. `read` 함수와 유사하게 `file_write`를 이용하여 작성된 바이트 수인 `write_size`를 계산하였으며, 동시에 일어나는 접근을 막기 위해 `lock`을 사용하여 원자적으로 처리하였다.

```
void sys_seek(int fd, unsigned position)
{
    struct file *f = (fd < thread_current()->fileCnt) ? thread_current()-
>fileTable[fd] : NULL;

    if (f != NULL)
        file_seek(f, position);
}
```

`sys_seek` 함수는 `fd`라는 파일 디스크립터를 가진 파일에서 읽거나 쓸 때 시작 위치를 `position`으로 변경하는 역할을 한다. `position`이 0이라면 파일의 시작점을 가리킨다. 만약 파일의 끝(EOF) 이후 위치를 가리키면, `read` 시 0을 반환하고, `write` 시에는 앞서 설명한 이유로 오류가 발생할 수 있다. 따라서 `position` 위치를 확인하는 과정이 필요하다. 이 함수는 `file_seek` 함수를 이용하여 구현하였다.

```
unsigned sys_tell(int fd)
{
    struct file *f = (fd < thread_current()->fileCnt) ? thread_current()-
>fileTable[fd] : NULL;

    return (f != NULL) ? file_tell(f) : 0;
}
```

`sys_tell` 함수는 `fd`라는 파일 디스크립터를 가진 파일의 다음 읽기 또는 쓰기 위치인 다음 바이트의 위치를 반환한다. 이 함수는 `file_tell` 함수를 이용하여 구현하였다.

```
void sys_close(int fd)
{
    struct file *f = (fd < thread_current()->fileCnt) ? thread_current()-
>fileTable[fd] : NULL;

    if (f != NULL) {
```



```

        file_close(f);
        thread_current()->fileTable[fd] = NULL;
    }
}

```

마지막으로 `sys_close` 함수는 `fd`라는 파일 디스크립터를 가진 파일을 닫는 역할을 한다. 프로세스가 종료될 때, 이 함수가 호출되어 모든 열려 있는 파일 디스크립터들이 자동으로 닫힌다. 이 함수는 `file_close` 함수를 호출하여 파일을 닫고, 해당 파일 디스크립터에 해당하는 `fileTable` 값도 `NULL`로 처리하였다.

```

bool verify_mem_address(void *addr)
{
    return addr >= (void *)0x08048000 && addr < (void *)0xc0000000 ;
}

```

위의 `syscall` 함수들에서 유효한 `address` 값을 가지고 있는지 확인하기 위해 사용된 함수이다. `0x08048000`은 `user` 영역의 시작주소를 나타내며, `0xc0000000`은 커널 영역의 시작주소를 나타낸다.

```

void getArgs(void *esp, int *arg, int count)
{
    for (int i = 0; i < count; i++)
    {
        if(!verify_mem_address(esp + 4 * i)){
            sys_exit(-1);
        }
        arg[i] = *(int *)(esp + 4 * i);
    }
}

```

위에 설명된 `system call` 함수들을 보면 인자의 개수가 다르다. 각 함수에서 올바른 인자를 넘겨주기 위해 `user stack`에 있는 값을 인자 개수만큼 가져와 `kernel`에 저장해야 한다. `getArgs` 함수는 해당 역할을 하며, 아래에서 설명할 `switch-case` 구조에서 각 `system call` 별로 호출될 예정이다.

```

static void
syscall_handler(struct intr_frame *f)
{
    if(verify_mem_address(f->esp))
    {
        int argv[3];
        switch (*(uint32_t *)(f->esp))
        {
            case SYS_HALT:
                sys_halt();
                break;
            case SYS_EXIT:
                getArgs(f->esp + 4, &argv[0], 1);
                sys_exit((int)argv[0]);
        }
    }
}

```

```
        break;
    case SYS_EXEC:
        getArgs(f->esp + 4, &argv[0], 1);
        f->eax = sys_exec((const char *)argv[0]);
        break;
    case SYS_WAIT:
        getArgs(f->esp + 4, &argv[0], 1);
        f->eax = sys_wait((pid_t)argv[0]);
        break;
    case SYS_CREATE:
        getArgs(f->esp + 4, &argv[0], 2);
        f->eax = sys_create((const char *)argv[0], (unsigned)argv[1]);
        break;
    case SYS_REMOVE:
        getArgs(f->esp + 4, &argv[0], 1);
        f->eax = sys_remove((const char *)argv[0]);
        break;
    case SYS_OPEN:
        getArgs(f->esp + 4, &argv[0], 1);
        f->eax = sys_open((const char *)argv[0]);
        break;
    case SYS_FILESIZE:
        getArgs(f->esp + 4, &argv[0], 1);
        f->eax = sys_filesize(argv[0]);
        break;
    case SYS_READ:
        getArgs(f->esp + 4, &argv[0], 3);
        f->eax = sys_read((int)argv[0], (void *)argv[1], (unsigned)argv[2]);
        break;
    case SYS_WRITE:
        getArgs(f->esp + 4, &argv[0], 3);
        f->eax = sys_write((int)argv[0], (const void *)argv[1], (unsigned)argv[2]);
        break;
    case SYS_SEEK:
        getArgs(f->esp + 4, &argv[0], 2);
        sys_seek(argv[0], (unsigned)argv[1]);
        break;
    case SYS_TELL:
        getArgs(f->esp + 4, &argv[0], 1);
        f->eax = sys_tell(argv[0]);
        break;
    case SYS_CLOSE:
        getArgs(f->esp + 4, &argv[0], 1);
        sys_close(argv[0]);
        break;
    default:
        sys_exit(-1);
    }
    }else{
        sys_exit(-1);
    }
}
```

이 함수는 switch-case 구조를 통해 각 system call을 구현하는 함수이다. 위에서 설명한 argument를 pop 해오는 함수 호출을 통해 인자를 argv stack에 저장하고, 위의 system call 함수 호출을 통해 handling한다. 반환값이 있는 경우, `f->eax`에 저장하도록 하였다.

Difference from design report

전체적인 로직은 기존 design report의 내용과 유사하나, 몇 가지 추가된 사항이 있다. 먼저, 기존의 design report에서는 인자를 저장하는 과정을 고려하지 않고 각 시스템 호출(syscall) 내부 함수만 구상하였으나, 새롭게 `syscall_handler`의 `switch-case`에서 사용자 스택에서 pop하여 인자를 전달하는 함수가 필요하여 구현하였다. 그리고 몇몇 시스템 호출 함수에서 사용자 영역에 접근하는지 확인하기 위해 `verify_mem_address` 함수를 추가 구현하여 잘못된 메모리 접근을 방지하였다.

3. Process Termination Messages

Implementation & Improvement from the previous design

이 과제에서는 user process가 종료될 때, process의 이름과 exit code를 출력하도록 구현하는 것이 목적이다. 이는 Pintos 문서에 설명된 대로 `exit` 함수를 호출할 때 `printf(...)`를 추가하여 구현할 수 있다. 따라서 위에서 설명한 syscall `sys_exit` 함수에 아래의 라인을 추가하였다.

```
void sys_exit(int status)
{
    thread_current()->exitCode = status;
    printf("%s: exit(%d)\n", thread_name(), status); // 3. Process Termination
    Messages
    thread_exit();
}
```

Difference from design report

기존에는 `thread` 구조체에 종료 여부를 확인할 수 있는 boolean 변수를 추가하는 방식을 고려했으나, `exit`을 처리하는 함수(`sys_exit`)에서 바로 `printf`를 실행해도 충분히 과제를 수행할 수 있음을 syscall을 구현하면서 깨달았다. 이에 따라 위와 같이 수정하였다.

4. Denying Writes to Executables

Implementation & Improvement from the previous design

마지막 과제는 열려 있는 파일에 쓰기 작업을 하지 않도록 하는 것이다. Pintos 문서에 나타난 대로 파일을 열 때 `file_deny_write` 함수를 함께 호출하여 쓰기를 제한하고, 파일을 닫을 때 `file_allow_write` 함수를 이용해 쓰기를 허용하는 과정을 추가하면 된다.

```
bool
load (const char *file_name, void (**eip) (void), void **esp)
{
    ...
    file = filesys_open (file_name);
```

```

...
// 파일 획득 성공시
t->fileExec = file;
file_deny_write(file); // 4. Denying Writes to Executables
lock_release(&FileLock);
...
}

```

먼저 `load` 함수에서 `filesys_open` 함수를 호출하기 때문에, 파일이 성공적으로 열렸다면 `file_deny_write` 함수를 호출하여 쓰기를 할 수 없도록 한다.

```

int sys_open(const char *file)
{
... // filesys_open 함수 성공 확인 이후
if (!strcmp(thread_current()->name, file)) {
    file_deny_write(f);
}
...
}

```

또한, 2번 과제에서 구현한 syscall 중 `sys_open` 함수에서도 `filesys_open` 함수를 호출하므로, 파일이 성공적으로 열렸다면 `file_deny_write` 함수를 호출하여 쓰기를 제한한다.

```

void
file_close (struct file *file)
{
    if (file != NULL)
    {
        file_allow_write (file);
        inode_close (file->inode);
        free (file);
    }
}

```

2번 과제의 syscall 함수 중 `sys_close` 함수에서 `file_close` 함수를 통해 파일을 닫는 동작이 이미 포함되어 있는데, 해당 함수를 살펴보면 `file_allow_write`가 이미 포함되어 있어 추가할 필요가 없다. 따라서 `load`와 `sys_open` 함수를 수정하는 것으로 이번 과제를 마무리할 수 있다.

Difference from design report

기존에는 `load` 함수에서 `file_deny_write` 함수를 호출하여 쓰기를 제한하고, 파일을 닫을 때가 아닌 `process_exit` 함수가 호출될 때 `file_allow_write` 함수를 호출하여 다시 쓰기를 허용하는 방식으로 설계하였다. 그러나 syscall을 구현하면서 `sys_open` 함수에서도 파일을 열어 쓰기를 제한하는 과정이 필요하며, 반대로 `process_exit` 대신 `file_close` 함수에서 이미 `file_allow_write` 함수를 사용하므로 별도로 쓰기 허용 작업을 추가하지 않아도 된다는 점을 알게 되어 이와 같이 수정하였다.

Overall Limitations

이번 과제는 기존 프로젝트에서 새롭게 정의해야 할 구조들이 많아서 특히 어렵게 느껴졌다. 각 스레드별로 PCB와 파일을 관리할 수 있도록 변수를 정의해야 했는데, 구현 과정에서 계속 변수가 추가되면서 수정을 자주 해야 하는 어려움이 있었다. 원래는 PCB와 파일을 별도의 구조체로 관리하려 했으나, syscall 부분에서 하위 클래스에 반복적으로 접근하다 보니 복잡해져, 최종적으로 스레드 구조체에 PCB와 파일 관리를 위한 변수를 모두 정의하는 것으로 수정하였다.

또한, 사용자 영역의 메모리에 접근하는 것이 올바른지 확인하고, race condition을 방지하기 위해 세마포어를 사용했는데, 어떤 함수에서 필수적인지 파악하는 데 시간이 많이 걸렸다.

마지막으로, 과제 4번에서 언급한 `file_close` 함수도 초기에는 design report의 잘못된 접근 방식을 시도하여 오류가 발생했으나, `file_close` 함수를 제대로 확인하여 과제를 마무리할 수 있었다.

Overall Discussion

이번 과제를 통해 PCB (Process Control Block)와 사용자 스택에 대해 이해할 수 있었다. 각 프로세스가 생성될 때마다 PCB가 할당되며, 이를 통해 부모-자식 프로세스 간의 관계를 유지할 수 있었다. 또한, PCB는 해당 프로세스가 실행 중인지 대기 중인지 등을 나타내기 위해 Boolean 변수와 세마포어(semaphore)를 사용하여 구현할 수 있었다.

그리고 이번 과제를 통해 사용자 스택의 메모리 관리 방식도 이해할 수 있었다. 사용자 스택은 주로 함수 호출 시 함수 인자와 반환 주소 등을 저장하는 역할을 하며, 스택의 가장 높은 주소부터 4바이트씩 감소하면서 각 인자와 반환 주소를 저장하는 구조를 배울 수 있었다.

마지막으로, 사용자 프로그램에서 발생하는 시스템 호출의 몇 가지 동작 방식과 파일 저장, 실행, 종료 방식에 대해서도 학습할 수 있었다.

지난 프로젝트에서는 스레드에 추가적인 구현을 모두 하나의 구조체로 묶어 관리하고자 manager라는 구조체를 구현하였으나 의도와는 달리 코드 구현상의 불필요한 복잡성이 증가하여 구현상의 편의를 위해 userprogram과 관련된 thread의 멤버들을 각자 단일한 멤버변수로 구현했다

Result

~pintos/src/userprog에서 make check한다. 아래와 같이 80pass가 나옴을 확인했다.


```
pass tests/filesys/base/syn-write
pass tests/userprog/args-none
pass tests/userprog/args-single
pass tests/userprog/args-multiple
pass tests/userprog/args-many
pass tests/userprog/args-dbl-space
pass tests/userprog/sc-bad-sp
pass tests/userprog/sc-bad-arg
pass tests/userprog/sc-boundary
pass tests/userprog/sc-boundary-2
pass tests/userprog/sc-boundary-3
pass tests/userprog/halt
pass tests/userprog/exit
pass tests/userprog/create-normal
pass tests/userprog/create-empty
pass tests/userprog/create-null
pass tests/userprog/create-bad-ptr
pass tests/userprog/create-long
pass tests/userprog/create-exists
pass tests/userprog/create-bound
pass tests/userprog/open-normal
pass tests/userprog/open-missing
pass tests/userprog/open-boundary
pass tests/userprog/open-empty
pass tests/userprog/open-null
pass tests/userprog/open-bad-ptr
pass tests/userprog/open-twice
pass tests/userprog/close-normal
pass tests/userprog/close-twice
pass tests/userprog/close-stdin
pass tests/userprog/close-stdout
pass tests/userprog/close-bad-fd
pass tests/userprog/read-normal
pass tests/userprog/read-bad-ptr
pass tests/userprog/read-boundary
pass tests/userprog/read-zero
pass tests/userprog/read-stdout
pass tests/userprog/read-bad-fd
pass tests/userprog/write-normal
pass tests/userprog/write-bad-ptr
pass tests/userprog/write-boundary
pass tests/userprog/write-zero
```

```
pass tests/userprog/write-stdin
pass tests/userprog/write-bad-fd
pass tests/userprog/exec-once
pass tests/userprog/exec-arg
```

```
pass tests/userprog/exec-bound
pass tests/userprog/exec-bound-2
pass tests/userprog/exec-bound-3
pass tests/userprog/exec-multiple
pass tests/userprog/exec-missing
pass tests/userprog/exec-bad-ptr
pass tests/userprog/wait-simple
pass tests/userprog/wait-twice
pass tests/userprog/wait-killed
pass tests/userprog/wait-bad-pid
pass tests/userprog/multi-recurse
pass tests/userprog/multi-child-fd
pass tests/userprog/rox-simple
pass tests/userprog/rox-child
pass tests/userprog/rox-multichild
pass tests/userprog/bad-read
pass tests/userprog/bad-write
pass tests/userprog/bad-read2
pass tests/userprog/bad-write2
pass tests/userprog/bad-jump
pass tests/userprog/bad-jump2
pass tests/userprog/no-vm/multi-oom
pass tests/filesys/base/lg-create
pass tests/filesys/base/lg-full
pass tests/filesys/base/lg-random
pass tests/filesys/base/lg-seq-block
pass tests/filesys/base/lg-seq-random
pass tests/filesys/base/sm-create
pass tests/filesys/base/sm-full
pass tests/filesys/base/sm-random
pass tests/filesys/base/sm-seq-block
pass tests/filesys/base/sm-seq-random
pass tests/filesys/base/syn-read
pass tests/filesys/base/syn-remove
pass tests/filesys/base/syn-write
All 80 tests passed.
make[1]: Leaving directory '/root/pintos/src/userprog/build'
root@a7173fe9bcc8:~/pintos/src/userprog#
```